

C Como Programar – 6ª Edição

Resoluções Detalhadas: Capítulo 2 – Exercícios Restantes

Paul Deitel (Assistente AI)

Exercícios Práticos - Capítulo 2 (Complemento)

Consolidando a Tomada de Decisão e Formatação de Saída

Exercício 2.18: Comparando inteiros

Objetivo: Ler dois inteiros e compará-los usando apenas a estrutura `if` de seleção única.

```
1  #include <stdio.h>
2
3  int main(void) {
4      int num1, num2;
5
6      printf("Digite dois inteiros: ");
7      scanf("%d%d", &num1, &num2);
8
9      if (num1 > num2) {
10         printf("%d é maior\n", num1);
11     }
12     if (num2 > num1) {
13         printf("%d é maior\n", num2);
14     }
15     if (num1 == num2) {
16         printf("Esses números são iguais\n");
17     }
18
19     return 0;
20 }
```

Exercício 2.20: Diâmetro, circunferência e área de um círculo

Objetivo: Trabalhar com constantes de ponto flutuante e o especificador `%f` dentro de `printf`, mesmo a variável `raio` sendo inteira (conforme o escopo do capítulo).

```
1  #include <stdio.h>
2
3  int main(void) {
4      int raio;
5
6      printf("Digite o raio do círculo: ");
```

```

7     scanf("%d", &raio);
8
9     /* A multiplicacao por um numero em ponto flutuante (como
10        3.14159 ou 2.0)
11        causa a conversao implicita do inteiro 'raio' para ponto
12        flutuante no calculo */
13     printf("Diametro: %f\n", 2.0 * raio);
14     printf("Circunferencia: %f\n", 2.0 * 3.14159 * raio);
15     printf("Area: %f\n", 3.14159 * raio * raio);
16
17     return 0;
18 }

```

Exercício 2.21: Formas com asteriscos

Objetivo: Desenhar formas formatando espaços e asteriscos.

```

1  #include <stdio.h>
2
3  int main(void) {
4      printf("*****\n");
5      printf("*****\n");
6      printf("*****\n");
7      printf("*****\n");
8      printf("*****\n");
9      printf("*****\n");
10     printf("*****\n");
11     printf("*****\n");
12     printf("*****\n");
13
14     return 0;
15 }

```

Exercício 2.22: O que o código imprime?

Resposta Analítica: O código `printf("\n**\n***\n****\n*****\n");` imprime um triângulo retângulo composto por asteriscos alinhados à esquerda. A sequência de escape `\n` força a quebra de linha após a impressão de cada grupo de asteriscos.

Exercício 2.23: Maiores e menores inteiros

Objetivo: Ler cinco inteiros e achar o maior e o menor usando seleção simples.

```

1  #include <stdio.h>
2
3  int main(void) {
4      int n1, n2, n3, n4, n5;
5      int maior, menor;
6
7      printf("Digite cinco inteiros: ");

```

```

8     scanf("%d%d%d%d%d", &n1, &n2, &n3, &n4, &n5);
9
10    maior = n1;
11    menor = n1;
12
13    /* Encontrando o maior */
14    if (n2 > maior) { maior = n2; }
15    if (n3 > maior) { maior = n3; }
16    if (n4 > maior) { maior = n4; }
17    if (n5 > maior) { maior = n5; }
18
19    /* Encontrando o menor */
20    if (n2 < menor) { menor = n2; }
21    if (n3 < menor) { menor = n3; }
22    if (n4 < menor) { menor = n4; }
23    if (n5 < menor) { menor = n5; }
24
25    printf("O maior valor é %d\n", maior);
26    printf("O menor valor é %d\n", menor);
27
28    return 0;
29 }

```

Exercício 2.25: Imprima as suas iniciais

Objetivo: Desenhar iniciais usando blocos das próprias letras. (Desenhando "P" e "D" para Paul Deitel).

```

1  #include <stdio.h>
2
3  int main(void) {
4      printf("PPPPPPPP\n");
5      printf("  P  P\n");
6      printf("  P  P\n");
7      printf("  P  P\n");
8      printf("  PP\n\n");
9
10     printf("DDDDDDDD\n");
11     printf("D  D\n");
12     printf("D  D\n");
13     printf(" D  D\n");
14     printf("  DDD\n");
15
16     return 0;
17 }

```

Exercício 2.26: Múltiplos

Objetivo: Determinar se o primeiro número é múltiplo do segundo usando o operador módulo (%).

Exercício 2.28: Faça a distinção entre erro fatal e não fatal

Resposta Analítica: Um erro **fatal** faz com que o programa trave e seja encerrado prematuramente (como tentar dividir por zero). Um erro **não fatal** permite que o programa continue rodando, mas com cálculos incorretos ou comportamento inesperado (um erro lógico, como somar em vez de subtrair).

Por que um erro fatal é preferível? Erros fatais falham ruidosamente; eles chamam a atenção do programador de imediato para a linha exata onde o programa quebrou. Erros não fatais (silenciosos) podem corromper dados, gerar relatórios de negócios incorretos ou tomar decisões equivocadas sem que ninguém perceba até que seja tarde demais. Na engenharia de software, preferimos que o sistema "falhe rápido" (fail-fast) em vez de operar com dados corrompidos.

Exercício 2.29: Valor inteiro de um caractere

Objetivo: Demonstrar que, internamente em C, caracteres são armazenados como números inteiros ASCII.

```

1  #include <stdio.h>
2
3  int main(void) {
4      /* Imprimindo os valores inteiros dos caracteres solicitados */
5      printf("A: %d\n", 'A');
6      printf("B: %d\n", 'B');
7      printf("C: %d\n", 'C');
8      printf("a: %d\n", 'a');
9      printf("b: %d\n", 'b');
10     printf("c: %d\n", 'c');
11     printf("0: %d\n", '0');
12     printf("1: %d\n", '1');
13     printf("2: %d\n", '2');
14     printf("$: %d\n", '$');
15     printf("*: %d\n", '*');
16     printf("+: %d\n", '+');
17     printf("/: %d\n", '/');
18     printf("Espaco em branco: %d\n", ' ');
19
20     return 0;
21 }
```